Zajęcia 4 — Obsługa długich sekwencji: LSTM, GRU i splot 1D

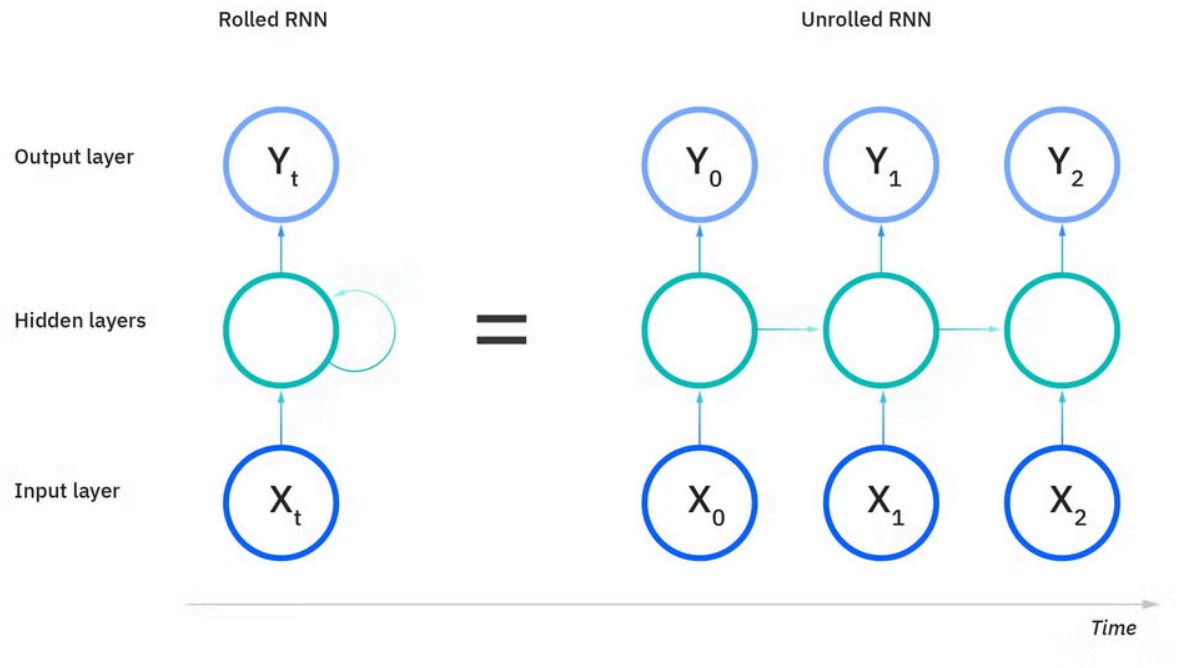
Blok: Fundamenty · **Sylabus:** L4

1. Dlaczego wracamy do RNN?

W prostym RNN każdy kolejny stan ukryty powstaje z bieżącego wejścia oraz poprzedniego stanu:

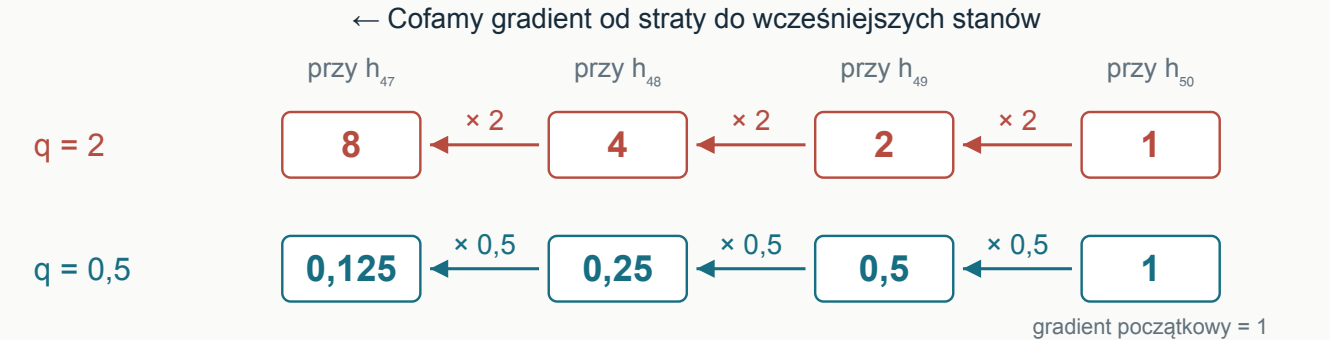
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

`x_t` oznacza wektor wejścia w chwili `t`, a `h_t` — wyuczoną reprezentację dotychczasowej sekwencji. Macierze wag są **współdzielone między krokami**. Rozwinięcie RNN w czasie pokazuje kolejne użycia tej samej komórki, a nie kolejne niezależne sieci.



Stan ma ograniczony rozmiar. Sieć nie zapisuje w nim automatycznie całej historii; musi nauczyć się zachowywać informacje potrzebne do zadania. Przykładowo po przeczytaniu długiego zdania może być potrzebny szczegół z jego początku, mimo że pomiędzy nim a odpowiedzią pojawiło się wiele innych słów.

W propagacji wstecznej przez czas sygnał błędu przechodzi przez kolejne przekształcenia. Reguła łańcuchowa wprowadza **iloczyny lokalnych pochodnych**: macierzy związanych z wagami i pochodnych aktywacji. Gdy ich łączny wpływ wielokrotnie zmniejsza sygnał, pojawia się zanikający gradient. Gdy go nadmiernie wzmacnia — gradient wybuchający.

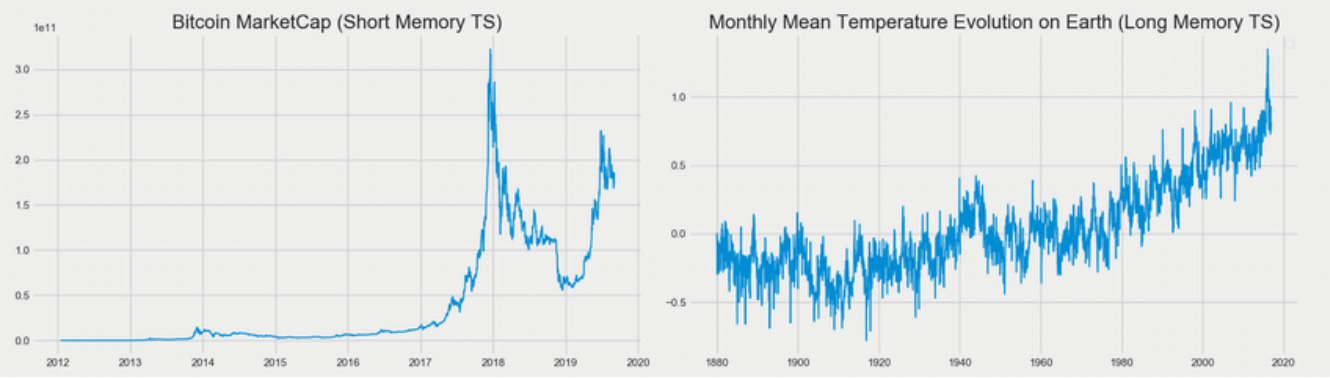


Skalary `0,5` i `2` to jedynie ilustracyjne mnożniki pochodnej na jednej drodze. Po 50 mnożeniach otrzymujemy odpowiednio około $8{,}9 \cdot 10^{-16}$ oraz $1{,}13 \cdot 10^{15}$. W rzeczywistej sieci nie występuje jedna stała liczba opisująca wszystkie te przekształcenia. **Clipping gradientu** ogranicza jego nadmierną normę, ale nie odzyskuje sygnału, który już zanikł.

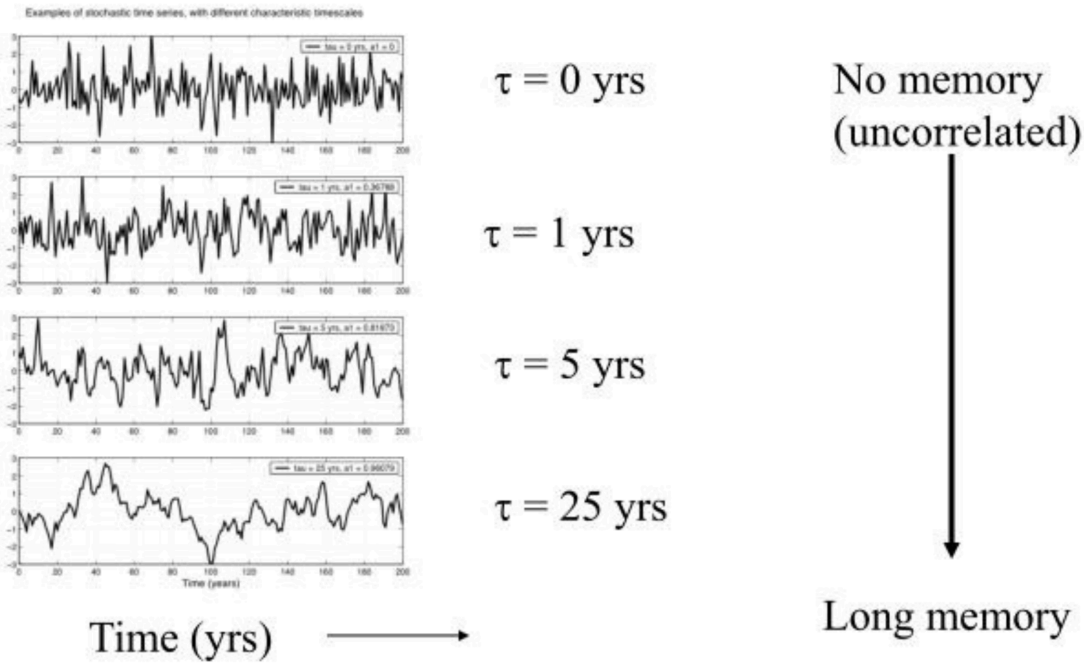
2. Co oznacza pamięć szeregu czasowego?

Intuicyjnie pytamy: **jak długo dawne obserwacje pozostają związane z późniejszym zachowaniem procesu?** Przy krótkim horyzoncie zależności wpływ przeszłości szybko słabnie. Przy dłuższym może być użyteczna wiedza o znacznie wcześniejszych wartościach. Nie chodzi o to, że sam szereg coś „zapamiętuje”; pamięć jest nazwą zależności statystycznych.

Poniżej pokazano wyłącznie górny wiersz dostarczonej ilustracji:



Etykiety na tym obrazku są opisem jego autora. Sam wygląd wykresu, trend czy powolna zmiana wartości **nie wystarczają, aby wykazać formalną długą pamięć**. Nie należy też utożsamiać jej z niestacjonarnością z poprzednich zajęć.



Na drugiej ilustracji **τ (tau)** oznacza charakterystyczny czas zaniku korelacji. Im jest większy, tym wolniej słabnie związek między oddalonymi obserwacjami. Jednostka τ jest jednostką czasu — tutaj są to lata. Dla przykładowego wykładniczego zaniku `ρ(Δt)=exp(−Δt/τ)` po czasie τ korelacja wynosi około 37% wartości początkowej. Podpis `τ=0` przedstawia przypadek bez korelacji między różnymi krokami.

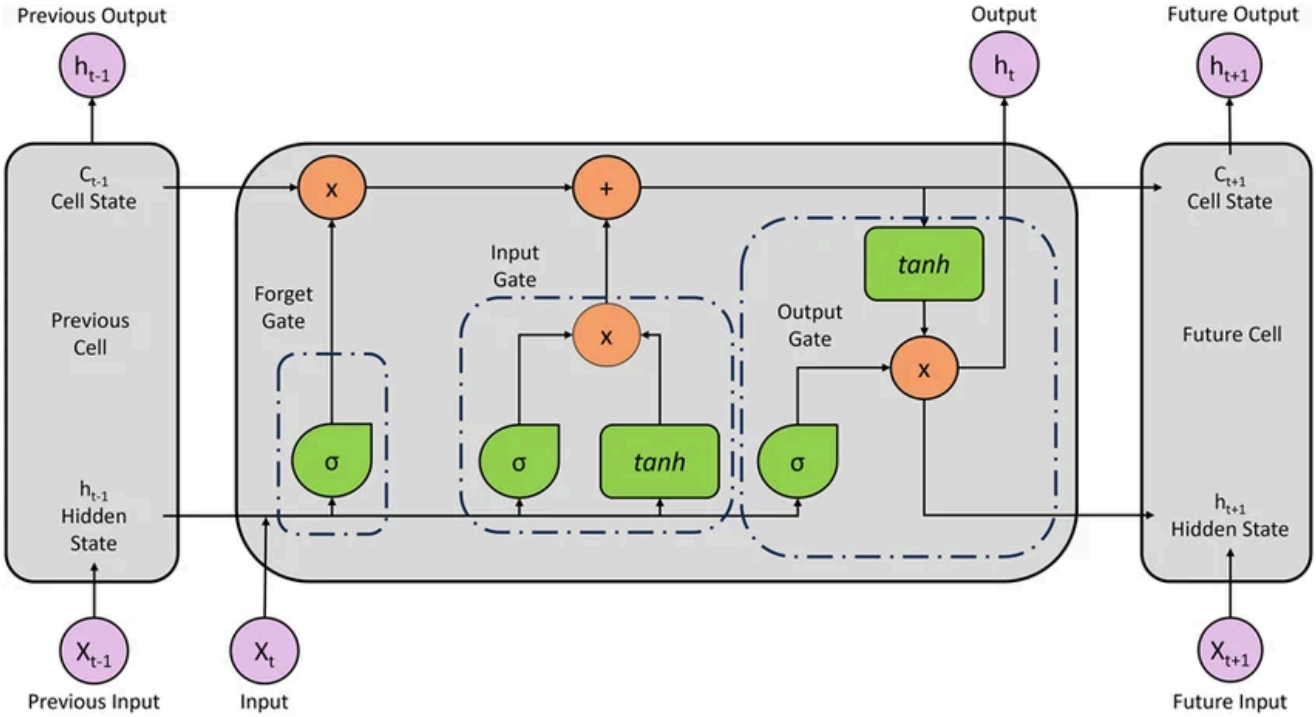
Duże, ale skończone τ ilustruje **dłuższy czas utrzymywania zależności**. Formalna długa pamięć wymaga odpowiednio wolnego zaniku zależności przy dużych opóźnieniach; proces z wykładniczym zanikiem i skończonym τ nadal ma krótką pamięć w tym ścisłym sensie. Na laboratorium potrzebujemy przede wszystkim intuicji: model może potrzebować informacji z kilku lub z wielu kroków wstecz.

Pamięć procesu i pamięć modelu to dwie różne rzeczy. Pierwsza opisuje dane, druga — zdolność architektury do zachowania i wykorzystania przydatnej informacji.

3. LSTM — pamięć sterowana bramkami

LSTM (Long Short-Term Memory) wprowadza dwa wektory przekazywane między krokami:

- **stan komórki** `c_t` — przechowywana reprezentacja informacji;
- **stan ukryty** `h_t` — reprezentacja udostępniana w bieżącym kroku kolejnym obliczeniom.



W górnej części schematu biegnie ścieżka stanu komórki. Bramki sterują tym, co zachować, co dopisać oraz co ujawnić. Każda bramka zwraca **wektor wartości od 0 do 1**, zwykle za pomocą sigmoidu. Nie jest to jeden przełącznik całej pamięci: każda składowa może otrzymać inny współczynnik.

W równaniach `⊙` oznacza mnożenie element po elemencie, `[h_{t−1},x_t]` — połączenie wektorów, `σ` — sigmoid. Każda bramka ma własne uczone wagi i bias. Taki zapis odpowiada standardowemu LSTM opisanemu w dokumentacji PyTorch.

Forget — ile starej pamięci zachować?

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f).$$

Na rysunku `f_t` jest sygnałem wychodzącym z zielonego bloku **σ w Forget Gate**. Pomarańczowe × na górnej linii mnoży go przez poprzednią pamięć `c_{t−1}`.

Wkład poprzedniej pamięci to `f_t ⊙ c_{t−1}`. Współczynnik `0,9` zachowuje 90% wartości danej składowej w jednym kroku; wartość bliska zeru silnie ją tłumi. Decyzja zależy od bieżącego wejścia i poprzedniego stanu ukrytego.

Input — co i ile dopisać?

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \qquad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c).$$

W ramce **Input Gate** wynik lewego σ to `i_t`, a wynik sąsiedniego tanh to `č_t`. Tylda oznacza proponowaną treść; pomarańczowe × łączy oba sygnały.

Kandydat `č_t` proponuje nową treść, a bramka `i_t` skaluje jej wkład. Kandydat nie jest kopią wejścia: to wyuczona reprezentacja o składowych z zakresu od −1 do 1. Aktualizacja pamięci łączy zachowaną część starego stanu z nowym wkładem:

$$\boxed{c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t}.$$

Dla jednej składowej: gdy `c_{t−1}=2`, `f_t=0,75`, `i_t=0,5`, `č_t=0,8`, nowa wartość wynosi `1,5 + 0,4 = 1,9`. Stan komórki nie jest ograniczony do przedziału −1..1. Bramki forget i input są niezależne, więc nie muszą sumować się do 1.

Output — ile pamięci ujawnić?

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \qquad h_t = o_t \odot \tanh(c_t).$$

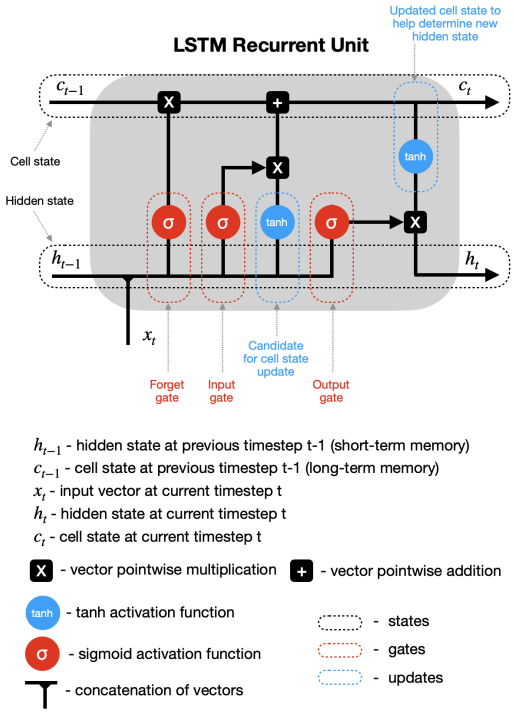
W ramce **Output Gate** dolny σ wytwarza `o_t`. Górny tanh otrzymuje stan `c_t`, a wynik końcowego × to `h_t`.

Najpierw `tanh` przekształca stan komórki, a potem bramka wyjścia skaluje jego ujawnienie w `h_t`. Jeśli `o_t` jest bliskie zeru, pamięć zostaje słabo ujawniona. **Nie oznacza to bezpośredniego wyzerowania** `c_t`, które jest przekazywane osobną ścieżką.

Dlaczego LSTM pomaga przy długich zależnościach?

Aktualizacja addytywna pozwala przenosić pamięć dalej i dopisywać do niej zmianę. W bezpośredniej gałęzi `c_{t-1} → c_t` lokalny mnożnik wynosi `f_t`. Gdy jest bliski 1 przez wiele kroków, sygnał może łatwiej przetrwać. To łagodzi problem zanikającego gradientu, **ale nie gwarantuje jego całkowitego usunięcia**. Powtarzane mnożenie nawet przez `0,9` nadal osłabia sygnał. Pełny gradient uwzględnia również inne drogi w grafie obliczeń.

Dodatkowy schemat z objaśnieniami:

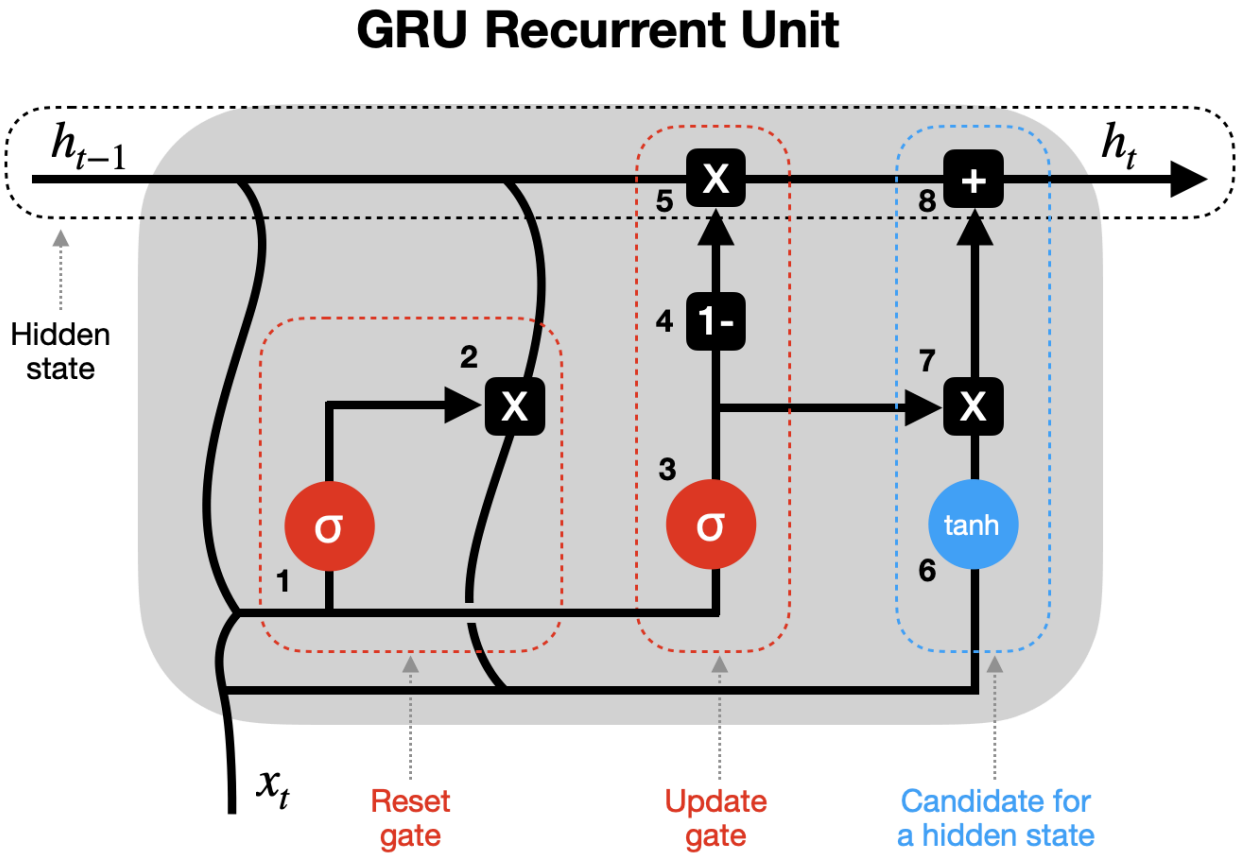


Ilustracja i materiał uzupełniający: LSTM — Towards Data Science.

4. GRU — jeden stan i dwie bramki

GRU (Gated Recurrent Unit) nie ma osobnego stanu komórki. Przechowuje jeden stan `h_t`, a jego aktualizację kontrolują dwie bramki:

- **reset** — ogranicza udział poprzedniego stanu przy obliczaniu nowego kandydata;
- **update** — ustala proporcje pomiędzy starym stanem a kandydatem.



Ilustracja i materiał uzupełniający: GRU — Towards Data Science.

Numery na diagramie odpowiadają trzem kolejnym etapom:

1. **Reset, kroki 1–2:** σ oblicza `r_t`, a × wykorzystuje go do przefiltrowania poprzedniego stanu na drodze do kandydata.
2. **Update, kroki 3–5:** osobny σ oblicza `u_t`; blok `1−` daje `1−u_t`, a × zachowuje taką część `h_{t−1}`. To gałąź starego stanu.
3. **Kandydat i połączenie, kroki 6–8:** tanh tworzy $\tilde{h}_t$ z wejścia i przeszłości przepuszczonej przez reset; × skaluje kandydata przez `u_t`, a + dodaje go do zachowanej części starego stanu.

Reset bliski zeru sprawia, że kandydat słabiej korzysta z przeszłości. Nie zeruje automatycznie całego nowego stanu, ponieważ stary stan dociera również osobną gałęzią do końcowego połączenia.

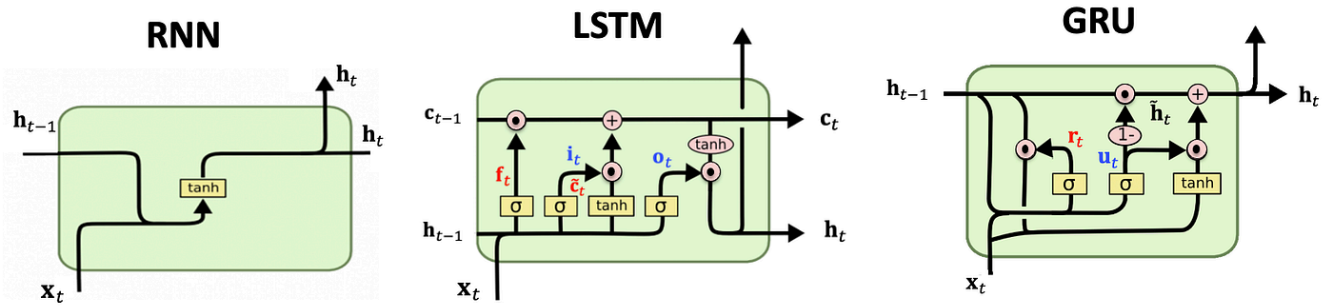
Na użytej ilustracji `u_t` jest udziałem **nowego** kandydata:

$$h_t = (1 - u_t) \odot h_{t-1} + u_t \odot \tilde{h}_t.$$

Wartość `u_t` bliska 1 oznacza większą zmianę, a bliska 0 — większe zachowanie starego stanu. Uwaga przy czytaniu kodu: PyTorch `nn.GRU` stosuje konwencję odwrotną — `z_t` jest udziałem **starego** stanu:

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot n_t.$$

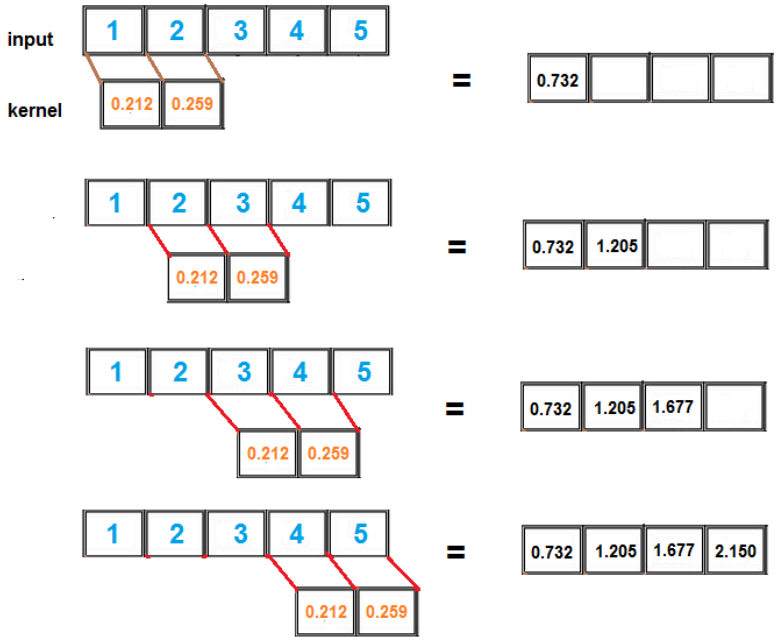
Te proporcje odpowiadają sobie przez `u_t=1−z_t`. Różni się też szczegół obliczania kandydata: w PyTorch reset mnoży przekształcony poprzedni stan, a nie stan przed jego przekształceniem liniowym. Schemat wyjaśnia ideę; przy implementacji trzeba trzymać się jednego wariantu równań.



Przy tym samym rozmiarze stanu GRU ma mniej parametrów niż LSTM. Nie przesądza to o błędzie predykcji ani czasie wykonania na danym urządzeniu. **Wybór pomiędzy nimi jest empiryczny.**

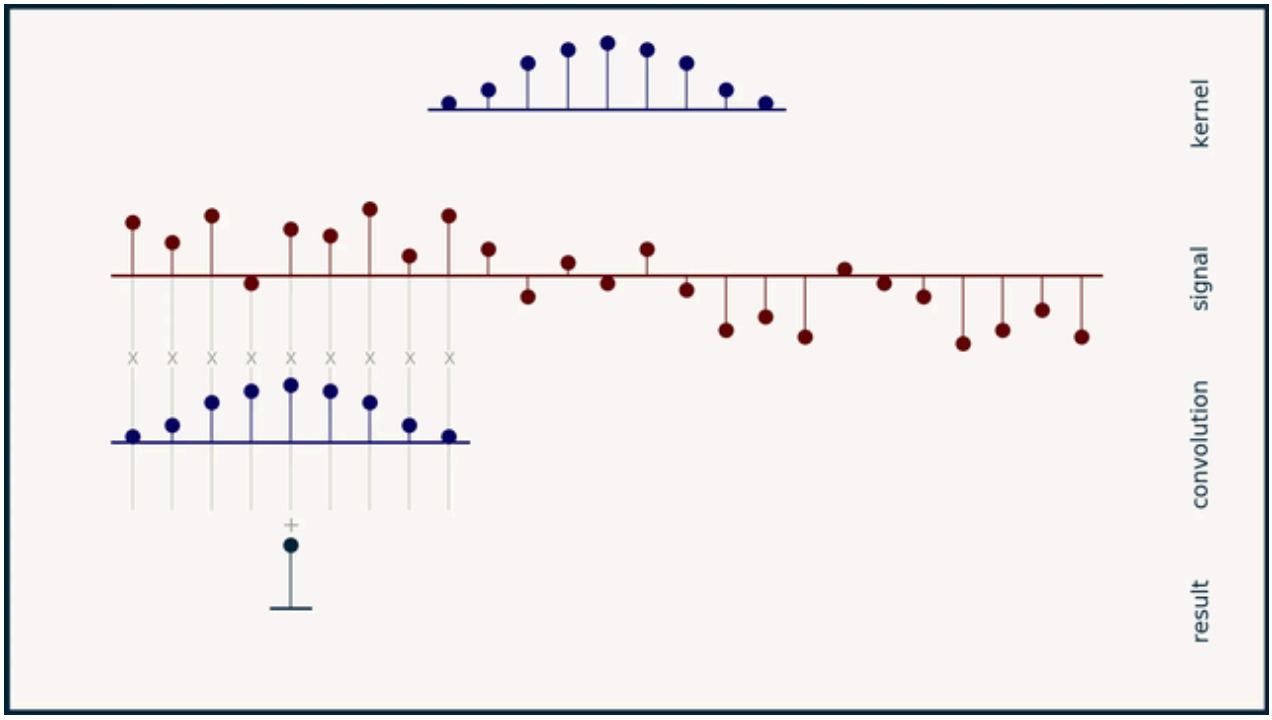
5. Konwolucja 1D — filtr przesuwany w czasie

Conv1D stosuje ten sam uczony filtr w wielu pozycjach sekwencji. Dla każdego fragmentu mnoży jego wartości przez wagi i sumuje wyniki, zwykle z dodaniem biasu. Przy wielu kanałach sumuje także po kanałach wejściowych. Pozwala wykrywać podobny lokalny wzorzec w różnych chwilach.



Ilustracja: Control and Learning. Na oryginalnej ilustracji wyniki liczbowe są niezgodne z wagami. Poprawny rachunek bez biasu dla pierwszego fragmentu: `1·0,212 + 2·0,259 = 0,730`.

`kernel_size` określa liczbę punktów filtra, **stride** — przesunięcie między kolejnymi pozycjami wyjścia, a **padding** — uzupełnienie brzegów. W PyTorch wejście Conv1D ma układ `[N,C,L]`: liczba przykładów, kanały, długość. Operacja nazywana splotem w `nn.Conv1d` jest technicznie korelacją krzyżową, bez odwracania jądra. Szczegóły: dokumentacja Conv1d.

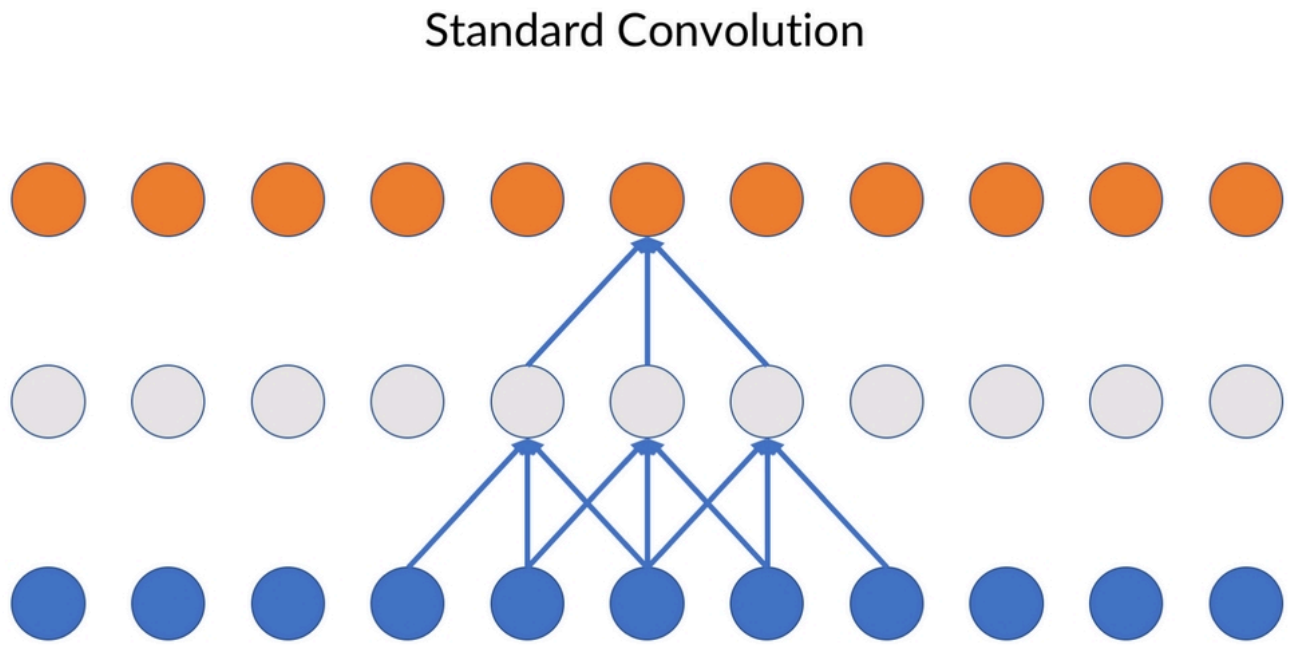


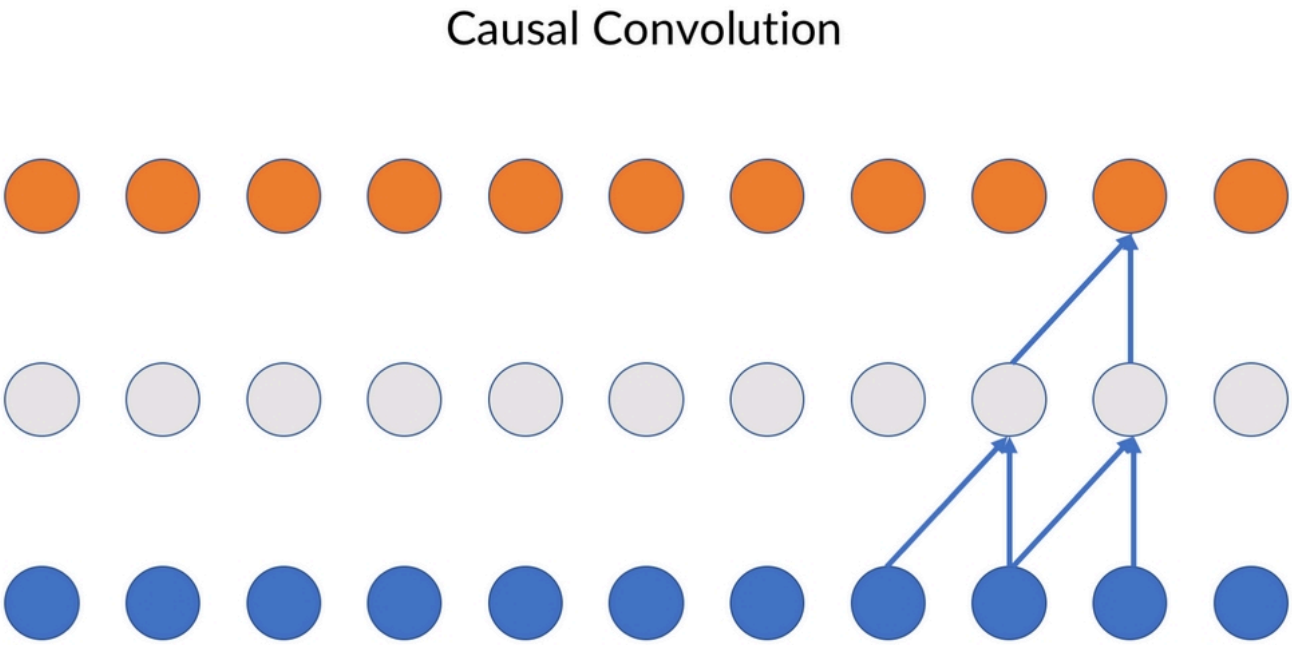
Animacje dla różnych kroków: Brandon Rohrer — Convolution in one dimension.

Standard, causal i dilated

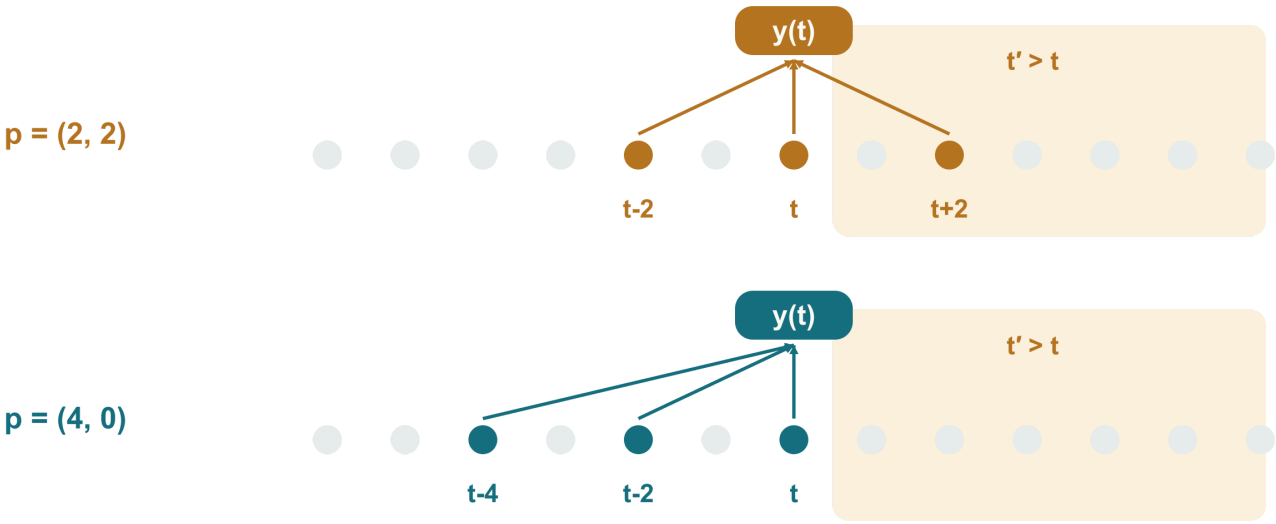
Te nazwy opisują różne właściwości i nie tworzą trzech rozłącznych kategorii.

Wariant	Intuicja	Na co uważać?
Standard, bez dylatacji	Filtr wybiera sąsiednie pozycje.	Symetryczne otoczenie może zawierać wartości późniejsze niż pozycja wyjścia.
Causal, przyczynowy	Wyjście w `t` nie korzysta z wejść późniejszych niż `t`.	Potrzebne jest odpowiednie wyrównanie filtra i paddingu.
Dilated, dylatowany	Punkty filtra są oddalone o `d` pozycji.	Dylatacja nie zapewnia przyczynowości; można łączyć obie właściwości.





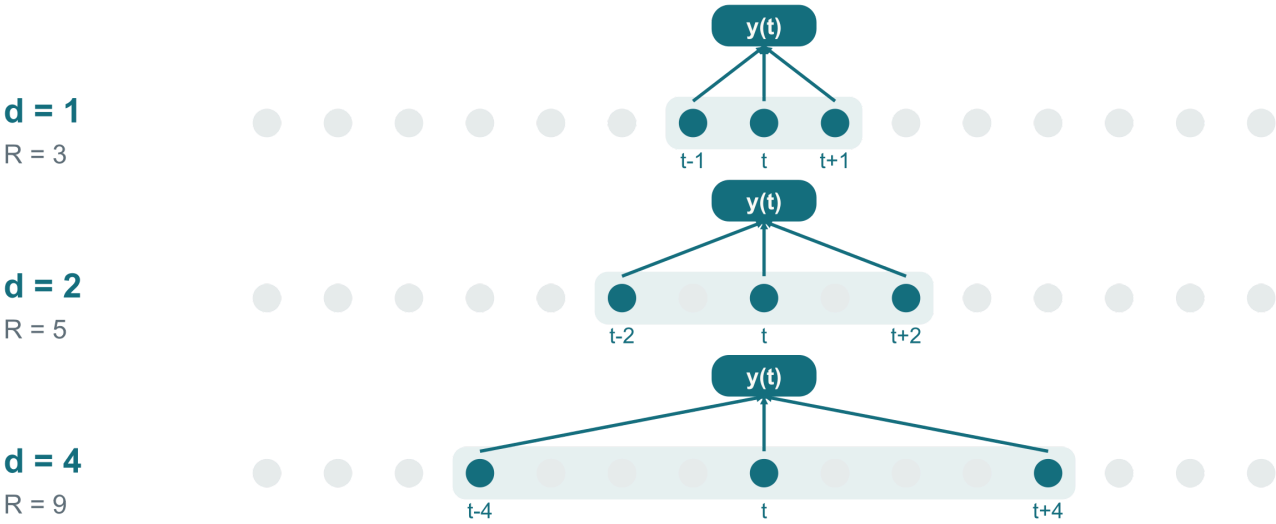
Na poniższym schemacie oba filtry mają trzy punkty i dylatację `d=2`. Górny korzysta z pozycji `t−2,t,t+2`. Dolny z `t−4,t−2,t`, więc jest przyczynowy.



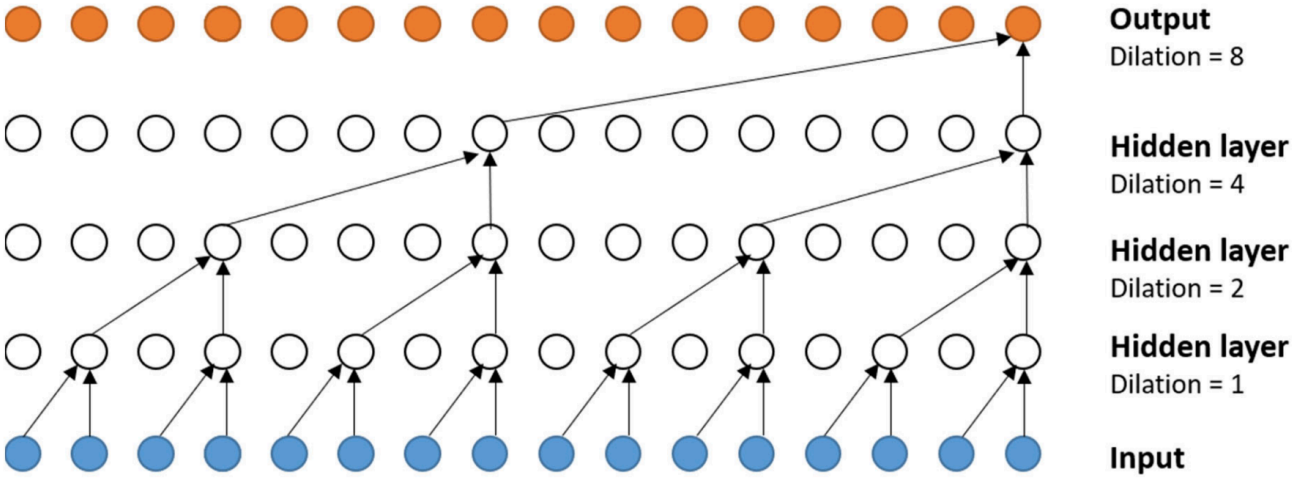
Przy prognozowaniu wolno używać tylko informacji dostępnej **w chwili wykonania prognozy**. Splot przyczynowy jest sposobem zachowania tego warunku dla wyjść w kolejnych krokach. Przy klasyfikacji już dostępnego całego nagrania symetryczny kontekst jest dopuszczalny.

Dylatacja i pole recepcyjne

Dilation określa odstęp między punktami jednego filtra. **Stride** mówi, o ile przesuwa się filtr, tworząc następny wynik. Dla trzech punktów filtra `d=1,2,4` daje rozpiętość odpowiednio `3,5,9` pozycji. Liczba wag pozostaje taka sama przy ustalonych kanałach i rozmiarze jądra.



Stos warstw zwiększa **pole recepcyjne**, czyli zakres wejścia wpływający na pojedynczą cechę wyjścia. Na dostarczonym diagramie jądro ma dwa punkty; warstwy z dylatacjami `1,2,4,8` obejmują razem 16 pozycji.



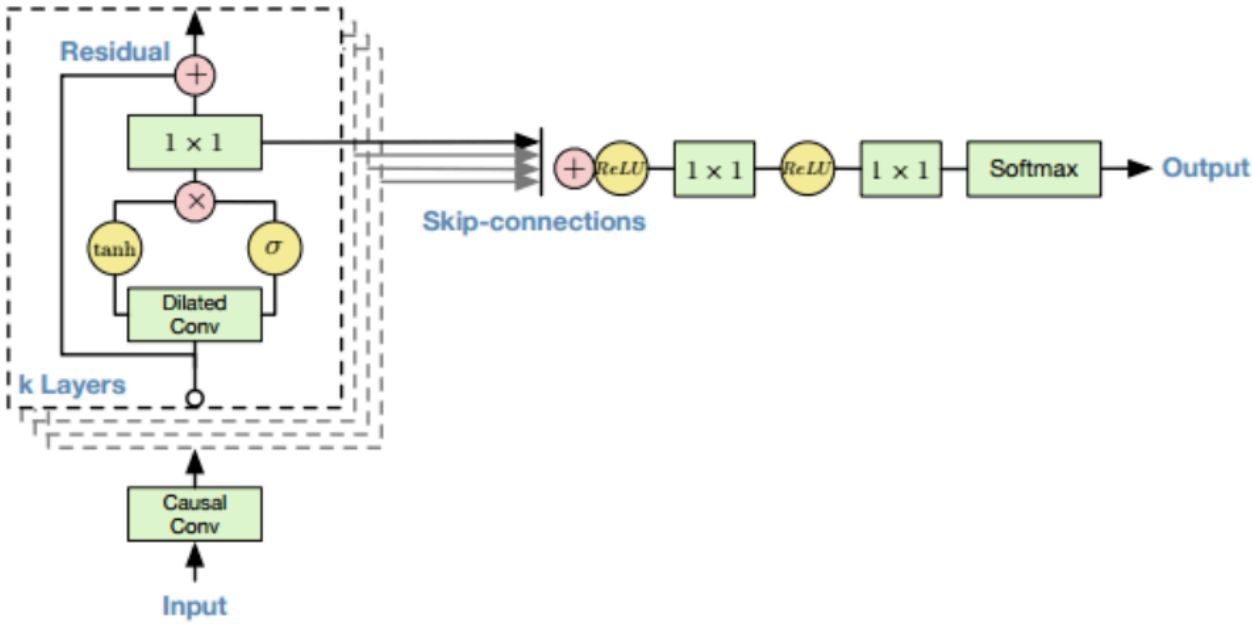
W notatniku jest inna konfiguracja: pięć warstw, `k=3`, `stride=1`, dylatacje `1,2,4,8,16`. Pole pojedynczej cechy końcowej mapy wynosi:

$$R = 1 + (k-1)\sum_{\ell} d_\ell = 1 + 2(1+2+4+8+16) = 63.$$

Wzór dotyczy tego stosu z krokiem 1. Na brzegach część pozycji wypełnia padding. Zwiększenie dylatacji nie dodaje wag do danej warstwy, ale **dodanie kolejnych warstw zwiększa łączną liczbę parametrów**.

ConvNet z notatnika ma symetryczny padding, więc nie jest przyczynowy. Po stosie splotów wykonuje `mean(dim=2)`, czyli uśrednia cechy po całej sekwencji. Dlatego **predykcja całego modelu może zależeć od całego wejścia**, mimo że pojedyncza cecha ma pole 63 pozycji.

WaveNet — związek z syntezą mowy



Ilustracja: NVIDIA OpenSeq2Seq. WaveNet (van den Oord i in., 2016) jest autoregresyjnym modelem surowego audio wykorzystującym rozszerzone sploty przyczynowe, bramki oraz połączenia residual i skip. Modeluje kolejną próbkę na podstawie wcześniejszych. Uczenie na znanej sekwencji umożliwia równoległe obliczenia pozycji, ale klasyczna generacja kolejnych próbek jest sekwencyjna. Nasz ConvNet jest prostym modelem regresji i nie stanowi implementacji WaveNet.

6. Adding problem — co mierzymy?

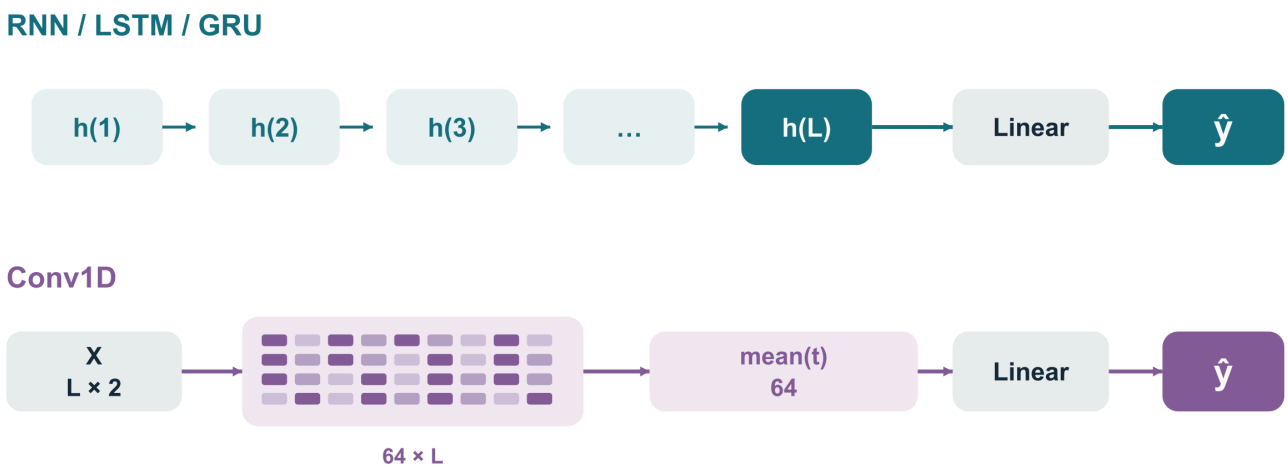
Wejście ma postać `[N,L,2]`. W każdym kroku są dwa kanały: losowa wartość `v_t∈[0,1]` i znacznik `m_t∈{0,1}`. Dokładnie dwa znaczniki wynoszą 1. Cel jest sumą wartości na tych pozycjach:

$$y = \sum_{t=1}^{L} m_t v_t.$$



Zadanie wymaga rozpoznania oznaczonych liczb i pominięcia pozostałych. W generatorze notatnika znaczniki są losowane bez wymuszania dużego odstępu. Dłuższe `L` dopuszcza bardziej odległe pozycje, ale nie oznacza, że każda próbka ma dwa dalekie znaczniki.

Odczyt wyniku ma znaczenie



RNN, LSTM i GRU przekazują ostatni stan ukryty do warstwy liniowej. Conv1D uśrednia całą mapę cech w czasie i dopiero wtedy oblicza wynik. Suma jest niezależna od kolejności składników: Conv1D może nauczyć się lokalnie rozpoznawać oznaczoną wartość i agregować ją globalnie. Nie musi koniecznie połączyć obu znaczników w jednym polu recepcyjnym. Benchmark **nie izoluje więc samej zdolności do pamiętania odległych kroków we wszystkich czterech architekturach**.

Stała predykcja jako punkt odniesienia

Każda z dwóch niezależnych wartości jednostajnych na `[0,1]` ma średnią `0,5` i wariancję `1/12`. Średnia celu wynosi 1. Model, który zawsze przewiduje 1, ma oczekiwany błąd:

$$\mathbb{E}[(y-1)^2] = \mathrm{Var}(y) = \frac{1}{6} \approx 0{,}167.$$

To **predykcja średniej**, a nie poziom losowego zgadywania. Na skończonym zbiorze testowym wynik będzie się nieco różnił. Niski test MSE oznacza, że model dokładniej odtwarza sumę; wynik bliski punktowi odniesienia sugeruje, że warto sprawdzić, czy nauczył się korzystać ze znaczników.

Porównanie architektur

Liczby parametrów poniżej obliczono z aktualnych klas notatnika przy rozmiarze stanu lub liczbie kanałów 64, z uwzględnieniem końcowej warstwy liniowej:

Model	Parametry
SimpleRNN	4 417
LSTM	17 473
GRU	13 121
Conv1D	49 921

Jednakowa szerokość nie zapewnia jednakowej pojemności ani kosztu. **MSE i czas trzeba zmierzyć**; nie zakładamy z góry rankingu architektur. Wynik zależy od długości, inicjalizacji, hiperparametrów, liczby epok i sprzętu.

Do kontrolowanego porównania używaj tych samych danych dla modeli, zapisuj ustawienia i urządzenie, a przy ocenie stabilności wykonuj powtórzenia z różnymi inicjalizacjami. Obecna funkcja `trenuj` losuje osobne dane dla każdego wywołania — to ograniczenie demonstracji, które należy uwzględnić w interpretacji. Dla ściśle sparowanego eksperymentu trzeba współdzielić dane lub kontrolować ich generator. Jeśli dobierasz ustawienia, użyj osobnej walidacji, a test pozostaw do końcowej oceny. Przy dokładnych pomiarach czasu na CUDA potrzebna jest synchronizacja urządzenia.

7. Ćwiczenie i pytania do wniosków

W notatniku uruchom cztery architektury dla co najmniej dwóch długości `L`. Zestaw test MSE, liczbę parametrów i czas uczenia. Zbadaj wpływ rozmiaru stanu ukrytego rekurencji oraz pola recepcyjnego Conv1D. Wymagania sprawozdania są na końcu notatnika i w zadaniu.

Przy interpretacji zapytaj: czy model pokonał stałą predykcję 1? Jak zmiana `L` wpłynęła na błąd i koszt? Czy poprawa wiązała się ze wzrostem liczby parametrów? Czy wynik Conv1D można wyjaśnić również globalnym uśrednianiem? Wnioski powinny wynikać z pomiarów, także wtedy, gdy nie potwierdzają oczekiwanej przewagi bramek.

W zastosowaniach ważne jest również, czy dane napływają w strumieniu, czy dostępne jest całe nagranie, i jaki kontekst jest dozwolony w chwili predykcji. LSTM/GRU aktualizują stan krok po kroku; Conv1D łączy lokalne wzorce w szerszy kontekst. Obie rodziny wymagają dopasowania sposobu przetwarzania i odczytu do zadania. Mechanizm uwagi na dalszych zajęciach pokaże kolejną drogę korzystania z odległych informacji.

Prowadzący: dr Tomasz Cieplak / mgr inż. Patryk Marek · Rok akademicki 2026/27